

Database :- It is a collection of data in format that can easily be accessed.

Why?

- can store large data
- features like security, scalability, etc
- Easier to insert, update or delete data (CRUD)

DBMS → Database Management System is a layer on database to manage data.

SQL

Relational Database
(data stored in tables)

e.g. MySQL, Oracle, PostgreSQL etc.

NoSQL

Non Relational database
(data stored in document/key, val/graph etc.) (not in tables)

e.g. MongoDB, Cassandra, No4j etc.

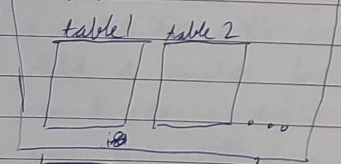
MySQL → Database

SQL → (Structured Query Language)

SQL is a programming language to interact with relational databases.

Entity 1

Database



inter-related

Entity 2

// Same

Table in SQL e.g

User				
ID	Name	Email	Followers	Following
1	Adam	adam@gmail.in	123	145
2	Bob	bob@gmail.com	200	200
3	Casey	casey@gmail.com	300	300
4	Donald	donald@gmail.in	200	105

columns → design (schema)

[id, name, email, followers, following]

rows → tuple

SQL is case insensitive. (Preferred Capital Letters)

- CREATE DATABASE db_name;
(To create new data base)
- DROP DATABASE db_name;
(To delete database)
- USE db_name;
(To specify the database, where table will be created)
- CREATE TABLE table_name (
column_name1 datatype constraint,
column_name2 datatype constraint,
column_name3 datatype, constraint,
); mandatory not "

e.g. create database

create table student (

e.g. create database college;

use college;

create table student (

roll_no int,
name varchar(30)

);

→ (to create schema/columns)

FOREIGN KEY: prevent actions that would destroy links b/w tables

It is basically used to link with other table's primary key

e.g. Link a math teacher's id with its students

- create table math teachers (
 - id int primary key,
 - t_name varchar (50),
 - age int

);

create table student (

- roll_no int,
- name varchar (50),
- t_id int,

foreign key (t_id) references math teacher(id)

);

Format → FOREIGN KEY (column1_name) REFERENCES table1_name (column1_name)

ER Diagram → Entity Relation Diagram (table)

What are Keys?

Keys are special columns in tables

Primary Key - It is a column (or a set of columns) in a table that uniquely identifies each row (a unique id). There is only 1 PK and it should be unique & not null.

Foreign key - It is a column (or set of columns) in a table that refers to the primary keys in another table. FKs can have duplicate & null values. There can be multiple FKs.

Table Queries: Insert into table →

INSERT INTO table_name (colname1, colname2) VALUES (col1_v1, col2_v1), (col1_v2, col2_v2);

Select Command: Selects & show data from the DB

- SELECT col1_name, col2_name from table
- SELECT col1_name, col2_name FROM table_name;
- SELECT * FROM table_name;
 - ↳ (to show all)
- SELECT DISTINCT col1_name FROM table_name;
 - ↳ (shows only unique values of col1_name)

Clauses - in SQL are similar to conditional in high-level language.

→ Where Clause: (to define some conditions)

• SELECT col1, col2 FROM table_name WHERE <condition>;

e.g. SELECT * FROM user WHERE age >= 30;

Operators that can be used with WHERE clause →

• Arithmetic Operators: +, -, *, /, %

• Comparison " " : =, !=, >, <=, <, >=

• Logical " " : AND, OR, NOT, IN, BETWEEN, ALL, LIKE, ANY

• Bitwise Operators: &, |

e.g. WHERE age + 1 = 18;

WHERE age > 15 AND followers > 200; (inclusive)

WHERE age BETWEEN 15 AND 20;] [15, 20]

WHERE age IN (14, 16);] only selects age 14, 16

WHERE name IN ("bob", "raj", "simon");

WHERE age NOT IN (14, 16);] selects every age except 14, 16

WHERE age NOT BETWEEN 10 AND 20;

LIKE - to search specified pattern

Two wild cards - % - Represents zero, one, or multiple chrs.
_ - Represents single character

e.g. WHERE name LIKE 'a%'; → selects all name starting with 'a'

WHERE name LIKE '%a'; → selects all name ending with 'a'

WHERE name LIKE '%a%'; → " " " that have 'a' in any position

WHERE name LIKE 'a_ _%'; → " " " starts with 'a' & is at least 3 chr in length

→ Limit Clause: (sets up an upper limit on number of (tuples) rows to be returned)

• SELECT col1, col2 FROM table_name
LIMIT (number);

e.g. SELECT name, age
FROM user

// WHERE age > 15 } multiple clauses
LIMIT 2;

→ Order by Clause: (to sort in ascending (ASC) or descending order (DESC))

• SELECT col1, col2 FROM table_name
ORDER BY col_name ASC;

e.g. ~~select~~ select name, age from user
ORDER BY age DESC;

Aggregate Functions: - perform a calculation on a set of values and return a single value.
(Pre defined functions)

• COUNT()

• MAX()

• MIN()

• SUM()

• AVG()

e.g. SELECT max(marks)
FROM student;

• SELECT count(age)

FROM user

WHERE age = 14; } Return count of students with age = 14

→ Group by Clause: (Groups rows that have the same values into summary rows)

It collects data from multiple records and groups the result by one or more column.

* (Generally we use group by with some aggregation fun.)

• SELECT col1, col2

FROM table_name

GROUP BY col_name(s);

e.g. SELECT age, count(id)
FROM user

GROUP BY age; } Select and groups unique age, & prints age with count of user with that age.

→ Having Clause: (Similar to where i.e. applies some conditions on rows)

But it is used when we want to apply condition after grouping (group by)

• SELECT col1, col2
FROM table_name
GROUP BY col_name(s)
HAVING <condition>;

- WHERE is for table, HAVING is for a group
- Grouping is necessary for HAVING

e.g. SELECT age, max(followers)
FROM user
GROUP BY age
HAVING max(followers) > 200;] First group
every unique age, then
find that age having maximum
followers of their age group. Then,
find age & maximum followers in
that age group.

General Order is:

• SELECT column(s)
FROM table_name
WHERE <condition>
GROUP BY column(s)
HAVING <condition>
ORDER BY column(s) ASC;

> Table Queries: Update (to update existing rows)
• UPDATE table_name
SET col1 = val1, col2 = val2
WHERE condition;

First time SQL will show error when we try to update
Use → • SET SQL_SAFE_UPDATES = 0;
(default is 1)

Now update will work.

> Table Queries: Delete (to delete existing rows)

• DELETE FROM table_name
WHERE <condition>;

e.g. [delete from user
where age = 30;] i.e. delete every row

> Table Queries: Alter (to change the schema)

Can alter column's: name, datatype, constraints, add, delete, etc.

→ ADD column

• ALTER TABLE table_name
ADD COLUMN column_name datatype constraint;

→ DROP column // to delete a column

• ALTER TABLE table_name
DROP COLUMN column_name;

→ RENAME Table

• ALTER TABLE table_name
RENAME TO new_table_name;

→ Change column (rename)

• ALTER TABLE table_name
CHANGE COLUMN old_name new_name new_datatype,
new_constraints;

or // rename column old_name to new_name;

must be same type
(if table column is not empty)

- MODIFY Column (modify datatype/constraint)
- ALTER TABLE table_name
MODIFY col_name new_datatype new_constraint;

> Table Queries: Truncate (to delete table's data)

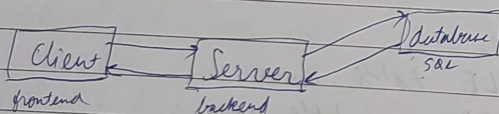
- TRUNCATE TABLE table_name;

↳ ~~DELETE~~ from
delete from table_name;] kind of same
both delete all data from the table
(rows)

- DROP table table_name;
- ↳ Deletes whole table.

Node with SQL:-

- npm install --save-dev @faker-js/faker
- ↳ Faker (@faker-js/faker) Package
Generates random data e.g. id, name, email,
image, password, dates, etc.



- npm i mysql2
- mysql2 package → connects node with MySQL
node.js →
const mysql = require('mysql2');

// now create connection to database

```
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  database: 'DB1', // database name
  password: 'JaiKam123' // MySQL password
}); // connection established
```

// now to run sql query →
try {

```
  connection.query('SHOW TABLES', // SQL query
    (err, results, fields) => {
      if (err) throw err;
      console.log(results);
    }
  );
```

```
}
catch (err) {
  console.log(err);
}
```

// connection object has method (function) .query() that
runs sql query.
connection.end(); // to close connection

Now we can run sql query using node.

To run sql on cmd/terminal →

- cd "C:\Program Files\MySQL\MySQL Server 8.0\bin"
- ↳ path of mysql

• mysql -u root -p
(Enter MySQL password) → Done
'quit' / '\q' / ctrl + D → to quit MySQL

~~cd~~

OR (From any location to
 "C:\Program Files\MySQL\MySQL Server 8.0\bin\
 mysql" -u root -p.
 (now enter password)
 // ~~to~~ *Only ~~with~~ * works with cmd, does not
 work with powershell

→ schema.sql // can run sql queries
 ↳ File

After entering sql workspace in cmd • write I
 mysql > source schem.sql;
 ↳ will run code that is written in
 schema.sql.

All ways to run ~~and~~ sql queries and access MySQL :-

- 1) Workbench
- 2) MySQL package → node.js
- 3) Cmd
- 4) using sql files + cmd

(All are synced, change in any will reflect in others also)

VS code

manually entering data using placeholder '?' and
 array.

Entering data (rows) ~~one~~ in table, one at
 a time v/s ~~multiple~~ multiple at a time.

Result in terminal

INSERT in Bulk using faker
 (data)

Routing with RESTful API :- (^{using} with MySQL database)

- GET "/" → shows no. of users in db
- GET "/user" → show users (email, id, username)
- PATCH "/user/:id" → username edit
- (ADD) POST "/user" → new user
- DELETE "/user/:id" → user delete (requires password)
- ↳ VS code

Mongo DB :- is NoSQL database.

The Shell :- (to manipulate db via terminal)

- mongosh // to start
- use "college" to create & use a new database
 called "college"

when start → temp. db → test

It's important there should be data in database
 otherwise it will get deleted from memory.

BSON Data :- (Binary JSON)

JSON

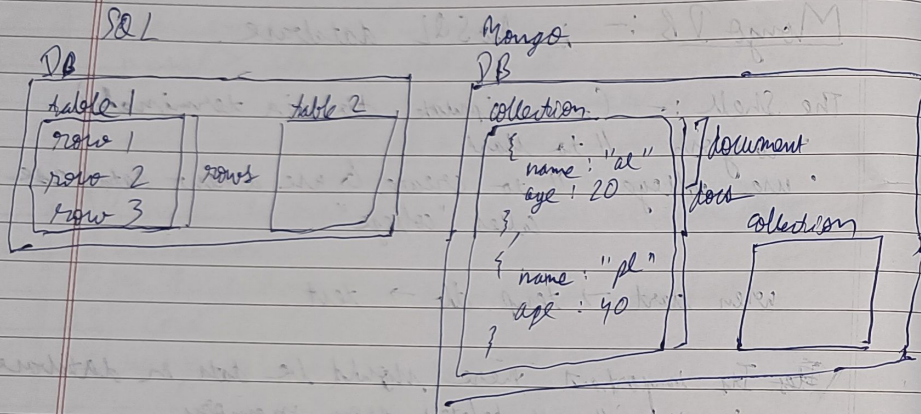
BSON

- | | |
|----------------------------------|--|
| - Text Based (UTF-8 String) | - Stores in binary |
| - Space inefficient | - efficient |
| - less data types | - extra datatypes e.g. Integer,
float, long, decimal 128, Array, null,
date, binary data |
| - Readability Human &
Machine | - Machine only |

We'll give JSON data, & Mongo will convert it into BSON.

Document :- Mongo stores data in form of documents (BSON docs)

Collection :- Mongo DB stores documents in collections. (collection of documents)



Insert in DB :-

- show collections (to show all collections in the db)
- show tables (also work)

Insert One (Insert a single document into a collection)

(terminal)

college>

db.student.insertOne({ name: "al", marks: 70 })

↳ returns acknowledged: true

and a unique uid which is primary key of this doc.

• db.student.find() (show all the docs in collection student)

we can give any key-value pair in any document unlike SQL rows (fixed schema / columns)

If a collection does not exist Mongo DB creates the collection when you first data for that collection.

insertMany (array of elements) (Insert multiple docs)

db.student.insertMany([{ name: "bob", age: 10 }, { name: "eve", city: "delhi" }])

Find in DB :- (Read)

db.collection.find() // returns array of docs in that collection

for specific queries

db.collection.find({ key: value })

db.collection.findOne({ key: value })
↳ (random single value)

- find returns cursor (array) (we can loop)
- findOne returns actual document.

multiple conditions db.student.find({ city: "delhi", marks: 75 })

Query Operators :-

0. Find students where marks > 75

db.student.find({ marks: { \$gt: 75 } })

0. Find students who live in delhi & mumbai

db.student.find({ city: { \$in: ["delhi", "Mumbai"] } })

0. Find students who scored > 75 or live in delhi

db.student.find({ \$or: [{ marks: { \$gt: 75 } }, { city: "delhi" }] })

Comparison 2

\$eq equals
 \$gt greater than
 \$gte " " equal
 \$in value in array
 \$lt less than
 \$lte " " equal
 \$ne not equal
 \$nin not in none of value in array

Logical 1

Element 2

\$and
 \$not
 \$or
 \$or
 \$exists: matches if field exists
 \$type: " " specified type
 {field: { \$exists: <boolean> }}
 {field: { \$type: <BSON type> }} → true/false
 or { \$type: [<type 1>, <type 2>, ...] }
 double/string/object/array/...

Update in DB:-

update One

db.collection.updateOne(1) Update at most a single document that match a specified filter even though multiple docs may match the specified filter.

db.collection.updateOne(<filter>, <update>, <options>)

e.g. for all name: "adam" set marks: 99

db.student.updateOne({ name: "adam" }, { \$set: { marks: 99 } })
 if field not there then creates new.

Update Operators 1 (to help with modification in the data/docs)

\$set → replaces the value of a field with the specified value (or creates if not there)

\$unset → deletes particular field

e.g. db.student.updateOne(
 { name: "bob" },
 { \$unset: { marks: " ", city: " " } }

db.collection.updateMany(1) Update all docs that match specified filter.

e.g. db.student.updateMany(
 { city: "delhi" },
 { \$set: { city: "New Delhi" } }

db.collection.replaceOne(1) Replaces doc that matches (complete new doc)

e.g. db.student.replaceOne(
 { name: "bob" },

(no need of \$set) { \$set: { name: "R.J", marks: 99 } }

Nesting :-

e.g. { name: "R.J"
 performance: { marks: 99, grade: 'A' } }

to find 2

db.student.find({ performance.marks: 99 })

nesting

Delete in DB:-

- delete One (`<filter>` `<options>`)
- delete Many (`<filter>`, `<options>`)

db.student.deleteMany({})] delete all docs

db.dropDatabase()] delete current db
(~~now~~ now db will be temporary)

Mongoose:-

A library that creates a connection b/w MongoDB & Node.js JS Runtime Environment.

It is ODM (Object Data Modeling) library.

- npm i mongoose

const ~~mongoose~~ mongoose = require("mongoose");

mongoose.connect("mongodb://127.0.1:27017/test");
(use async fun.) fined dis
similar to http url

```
async function main() {
  await mongoose.connect(" ", " ");
}
```

main().

```
.then(() => { console.log("connection success"); })
.catch(err => console.log(err));
```

Schema:- (overall structure)

" defines the shape of docs within that collection.

```
const user userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
```

Models:-

" in mongoose is a class with which we construct documents

```
const User = mongoose.model("User", userSchema);
```

↓ model name / class ↓ collection name
usually same name

JS - MongoDB (changes name)
 User → Users
 Product → products

Insert:-

Model → class → To create Objects (docs)

Inserting One

```
const user1 = new User({ name: "Adam", email: "adam@gmail.com", age: 48 });
```

```
const user2 = { name: "Eve", email: "eve@ ", age: 48 };
```

Promise
 [user1.save(); // to save in DB
 user2.save();]


```

Insert Many
User.insertMany([
  { name: "any", email: "", age: 50 },
  { name: "Aren", email: "", age: 50 }
]).then(res => console.log(res));

```

Note :- Mongoose uses Operation Buffering
Mongoose lets you start using your models immediately, without waiting for mongoose to establish the connection to MongoDB.

Find :-

```

model.find() // returns a Query Object (thennable)
* Mongoose Queries are not Promise. But they have a .then()

```

```

User.find().then(data => {
  console.log(data)
});

```

• findOne(<condition>) // returns One data

• findById("<Id string>").then ---

Update :-

```

Model.updateOne()

```

```

User.updateOne({ name: "Bruce", { age: 49 } },
  then(res => console.log(res));

```

```

Model.updateMany()

```

```

User.updateMany({ age: { $gt: 40 } }, { age: 45 },
  then ---

```

```

User.findOneAndUpdate({ name: "Bruce", { age: 45,
  { new: true } }, then ---
  ↳ optional / returns modified data

```

```

User.findOneBy Id And Update ("<id string", "

```

Delete :-

```

Model.deleteOne() // returns count
User.deleteOne({ name: "Adam" }).then(res =>
  console.log(res));

```

```

User.deleteMany({ age: { $gt: 40 } }). ---

```

```

Model.findByIdAndDelete()
Model.findOneAndDelete()

```

C	R	U	D	(in Mongo / Mongoose)
create	read	update	delete	
↓	↓	↓	↓	
insert	find	update	delete	

Schema Validations :- (Constraints / Rules)

basically, Rules for schema
e.g. const bookSchema = new mongoose.Schema({
 title: {
 type: String,
 required: true } // constraints
});

parse/cast → convert one datatype to another

Schema type Options :-

- type → String, Number, Date, ~~bool~~ boolean, Array, ...
- Required → true/false
- default → Set default value

Options →

- type → All → required, default, immutable, etc.
- String → lowercase, uppercase, trim, minlength, maxlength, enum
 (enum → only in given array)
 e.g. category: { type: String, enum: ["fiction", "non-fiction"] }
- Number → min, max, enum, ...
- Date → min, max, ...

- Normal Update will not follow validations, directly update. Validations only work at insertion not updation.

But there is a special option for validation in updation: { runValidators: true }

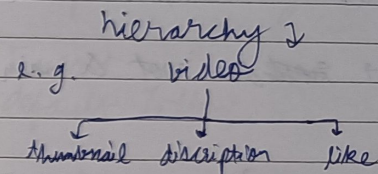
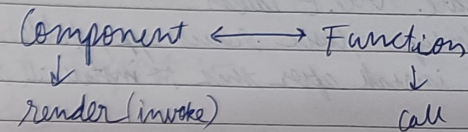
e.g. Book.findByIdAndUpdate({
 " id "
 price: 100
 runValidators: true
 })

define custom error + price: { type: Number, min: [1, "Price is too low"] }

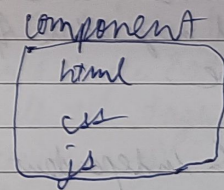
MongoDB with Express :- (VS Code)

React :-

JS Library for creating UI. Created and by facebook.



React is about building a component then reusing it again & again. be as small as button and as large as...



JSX :- JavaScript Extension
 It lets us write HTML directly in JS.
 It is not real JS.

